TP11 Gestión de Calidad — Resolución

1. Requerimientos

El programa debe pedir un entero por teclado, validar que esté entre 1 y 1999, calcular las iteraciones de Collatz hasta llegar a 1 e imprimir el número de partida y las iteraciones. Siguiendo la consigna, R = funcionales y F = no funcionales.

ID	Tipo	Requerimiento
R1	Funcional	Solicitar y aceptar por teclado un número entero positivo
R2	Funcional	Validar la entrada: entero en [1, 1999]; rechazar con mensaje claro 0, negativos, decimales, texto y entrada vacía
R3	Funcional	Calcular las iteraciones: si es par dividir entre 2, si es impar 3n+1, hasta llegar a 1 (n=1 requiere 0 iteraciones)
R4	Funcional	Imprimir el número de partida y el número de iteraciones
F1	No funcional	Límite práctico: el máximo entero a validar es 1999
F2	No funcional	Implementación en Python 3, sin dependencias externas
F3	No funcional	Rendimiento: respuesta en menos de 1 s para cualquier entrada válida
F4	No funcional	Robustez: ninguna entrada produce traceback ni bucle infinito; los errores van a stderr con prefijo `Error:` y código de salida distinto de 0

R = {R1, R2, R3, R4} y F = {F1, F2, F3, F4}.

2. Casos de test y matriz de trazabilidad

Son 16 casos: 8 unitarios (caja blanca, sobre `collatz()`) y 8 funcionales (caja negra, ejecutando el programa con entrada por teclado). Los marcados con "sesión 2" se agregaron durante la inspección de los tests.

ID	Tipo	Caso	Resultado esperado	Origen
T1	Unitario	`collatz(1)`	0 (no entra al `while`)	inicial
T2	Unitario	`collatz(2)`	1 (solo rama par)	inicial
T3	Unitario	`collatz(3)`	7 (ramas impar y par)	inicial
T4	Unitario	`collatz(27)`	111 (secuencia larga, pico 9232)	inicial

ID	Tipo	Caso	Resultado esperado	Origen
T5	Unitario	`collatz(0)`	`ValueError` (evita bucle infinito)	inicial
T6	Unitario	`collatz(2000)`	`ValueError`	inicial
T7	Unitario	`collatz(1999)`	50 (límite superior válido)	sesión 2
T8	Unitario	`collatz(-5)`	`ValueError`	sesión 2
T9	Funcional	entrada `20`	"El número de iteraciones para 20 es 7"	inicial
T10	Funcional	entrada `2000`	error limpio en stderr, código ≠ 0, sin traceback	inicial
T11	Funcional	entrada `-5`	ídem	inicial
T12	Funcional	entrada `abc`	ídem	inicial
T13	Funcional	entrada `1999`	"... para 1999 es 50"	sesión 2
T14	Funcional	entrada `3.5`	ídem T10	sesión 2
T15	Funcional	entrada vacía (EOF)	ídem T10	sesión 2
T16	Funcional	entrada `1999` con medición de tiempo	responde en menos de 1 s	sesión 2

Matriz RTMX ($R \cup F \Leftrightarrow T$)

Req	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15
R1									x				x		x
R2					x	x		x		x	x	x		x	x
R3	x	x	x	x			x		x				x		
R4									x				x		
F1						x	x			x			x		
F2									x				x		
F3															
F4					x			x		x	x	x		x	x

Validación de integridad: ningún requerimiento queda sin casos (cada fila tiene al menos una marca) y ningún caso queda sin requerimiento (cada columna tiene al menos una marca). La matriz se actualizó al agregar T7, T8 y T13–T16 en la sesión 2.

3. Inspecciones

La inspección del código (sesión 1) detectó 7 defectos y la de los casos de test (sesión 2) detectó 5, en ambos casos sin ejecutar nada. El código original tiene 2 líneas de comentario sobre 18 Locs (11,1 %), por debajo del umbral de 12 que la consigna fija como

defecto, así que no suma uno.

Sesión 1: inspección del código

#	Lugar	Tipo	Defecto	Afecta
1	`print(...)`	Sintáctico	Falta el paréntesis de cierre: el archivo no compila	todos
2	`collatz(j)`	Semántico	`j` no está definida (`NameError`); la variable es `i`	R4
3	`return iter`	Semántico	Devuelve la función `iter` en lugar de `iteraciones`	R3
4	`i=20`	Semántico	Valor fijo: nunca se pide el número por teclado	R1
5	`collatz`	Semántico	Sin validación de rango: 0 y negativos generan un bucle infinito y 2000 o más se acepta	R2, F1, F4
6	entrada	Semántico	Sin control de entradas no numéricas, decimales o vacías	R2, F4
7	general	Estilo	Sin docstrings ni función principal, e indentación de 2 espacios (PEP 8 pide 4)	mantenibilidad

Sesión 2: inspección de los casos de test

#	Defecto del conjunto de pruebas	Caso agregado
1	Falta el límite superior válido (1999)	T7, T13
2	Falta un valor negativo en las pruebas unitarias	T8
3	Falta una entrada decimal	T14
4	Falta la entrada vacía (EOF)	T15
5	Nada verifica el requisito no funcional F3 (rendimiento)	T16

La matriz RTMX de la sección 2 ya incluye estos casos.

4. Sesiones de ejecución

Las 8 sesiones se ejecutaron de verdad (script `run_sesion.py` , cada caso en un subproceso con timeout de 5 s) y arrojaron 18 defectos: 6 en las sesiones unitarias S1–S5 y 12 en las funcionales S6–S8. Entre sesiones se repararon todas las fallas observadas y nunca se hicieron corridas individuales por falla.

Sesión	Tipo	Versión	Defectos	Qué falló	Reparación antes de la siguiente
S1	Unitaria	v0 (original)	1	`SyntaxError` al importar: paréntesis sin cerrar	cerrar el paréntesis
S2	Unitaria	v1	1	`NameError`: `j` no definida	usar `i`
S3	Unitaria	v2	1	`TypeError`: `%d` recibe la función `iter`	`return iteraciones`
S4	Unitaria	v3	3	T5 y T8: bucle infinito (timeout); T6: devuelve 112 sin error	validar el rango 1 a 1999
S5	Unitaria	v4	0	8 de 8 casos exitosos	ninguna
S6	Funcional	v4	7	T10 a T16: el programa no lee el teclado e imprime siempre el caso 20 (T9 pasó por coincidencia)	agregar `main` con `input()`
S7	Funcional	v5	5	T10, T11, T12, T14, T15: traceback (`ValueError`, `EOFError`) en lugar de mensaje	`try/except` con `Error:` en stderr y código 1
S8	Funcional	v6	0	8 de 8 casos exitosos	ninguna

La versión final v6 también pasa los 8 casos unitarios (regresión) y mide 45 líneas.

Listado de sesiones y defectos

Sesión	I1	I2	S1	S2	S3	S4	S5	S6	S7	S8	Total
Defectos	7	5	1	1	1	3	0	7	5	0	30

El total es de 10 sesiones (2 de inspección, 5 unitarias y 3 funcionales) con 30 defectos.

Verificación de condiciones finales

- Todos los casos de prueba (T1 a T16) se ejecutaron con éxito al menos una vez: T1 a T8 en S5 y T9 a T16 en S8.
- No quedó ningún caso sin ejecutar.
- Ningún requerimiento (R1 a R4, F1 a F4) quedó sin verificar, según la matriz RTMX.

Las inspecciones no repararon el código, así que las fallas de causa raíz ya señaladas en ellas se volvieron a observar al ejecutar. Por eso los 30 defectos cuentan cada fase por separado, como pide la consigna; sin duplicados hay 7 causas raíz en el código y 5 omisiones en los tests.

5. PCE por fase y densidad de defectos

El PCE acumulado llega al 40 % tras la inspección, al 60 % tras las pruebas unitarias y al 100 % tras las funcionales, tomando como 100 % los 30 defectos observados. El PCE de cada fase es lo que encontró dividido por lo que había presente al entrar.

Fase	Defectos encontrados	Presentes al entrar	PCE de la fase	PCE acumulado
Inspección (I1 + I2)	12	30	40,0 %	40,0 %
Unitario (S1 a S5)	6	18	33,3 %	60,0 %
Funcional (S6 a S8)	12	12	100 %	100 %

Densidad de defectos al comienzo del test, con S = 18 Locs: 30 / 18 = 1,67 defectos por Loc (1667 por KLoc). Si se cuenta solo desde que arranca la ejecución, después de la inspección, son 18 / 18 = 1,0 defecto por Loc.

6. Kanban: parámetro TEP

TEP es el Trabajo En Proceso (WIP en inglés): la cantidad de trabajo iniciado y todavía no terminado, y el valor crítico de la metodología es el límite máximo de TEP que el sistema admite. Una fuente académica sobre Kanban describe su objetivo justamente como minimizar el TEP, o el stock entre procesos (Universidad de Medellín, Redalyc).

- **Cómo funciona:** en un sistema de arrastre (pull) solo se empieza un ítem nuevo cuando se libera lugar en el tablero; al llegar al límite se deja de iniciar y el equipo se concentra en terminar lo que está abierto.
- **Por qué importa:** por la ley de Little, tiempo de ciclo = TEP / tasa de salida. Con la misma capacidad, más TEP no entrega más rápido: solo alarga las esperas y esconde cuellos de botella. En software también retrasa la detección de defectos.
- **Significado crítico del valor:** si el límite es demasiado alto, se forman colas y el control se pierde; si es demasiado bajo, hay gente sin trabajo y cae el rendimiento. Se fija en el punto que mantiene ocupado al cuello de botella sin acumular cola y se ajusta con mediciones de tiempo de ciclo y rendimiento.

No pude acceder al tutorial "Kanban para uso genérico" del Ing. Ruiz de Mendarozqueta, así que esta explicación sale de la definición general de TEP y de la ley de Little. Conviene contrastarla con cómo lo define el tutorial.

7. Regresión sobre el dataset del TP

Los datos no muestran una tendencia decreciente clara, así que el total proyectado es de unos 120 defectos o más, con confianza baja. Los defectos diarios de los días 0 a 6 fueron 9, 18, 5, 7, 23, 2 y 8 (suma 72, promedio 10,3 por día).

Modelo	Ajuste	R^2	p de la pendiente	Total proyectado
A. Lineal sobre defectos diarios	$d = 12{,}11 - 0{,}61 \cdot t$	0,03	0,71	126,8 (cruza 0 hacia el día 20)
B. Log-lineal sobre defectos diarios	$\ln d = 2{,}42 - 0{,}115 \cdot t$	0,09	0,50	118,4

Modelo	Ajuste	R^2	p de la pendiente	Total proyectado
C. Acumulado saturante	$D(t) = \mu0 \cdot (1 - e^{-k(t+1)})$ con $\mu0 = 303,7$ y $k = 0,040$	0,96	n/a	303,7

Criterios para evaluar la proyección: que la pendiente sea negativa (si no hay decaimiento no existe total finito), su significancia estadística, el R^2 sobre los datos diarios y la estabilidad del total entre modelos.

- Los modelos A y B cumplen el primer criterio pero no el resto: R^2 de 0,03 y 0,09 y p mayor a 0,5, es decir la pendiente no se distingue de cero.
- El modelo C tiene R^2 alto solo porque el acumulado siempre sube. Con $k = 0,04$ la curva es casi una recta (unos 10 defectos por día sin frenar), así que su 304 es una extrapolación y funciona como cota superior.

Conclusión: se esperan al menos unos 120 defectos en total, es decir 46 a 55 más que los 72 ya encontrados, y no hay evidencia de que el ritmo esté bajando.

8. Regresión sobre las sesiones propias

Con los resultados de este TP, el modelo de mejor ajuste proyecta unos 57 defectos totales, de los cuales 27 seguirían latentes al terminar el test. Se asimila "Día" a "Sesión" y la serie es I1, I2, S1 a S8: 7, 5, 1, 1, 1, 3, 0, 7, 5, 0 (suma 30).

Modelo	Ajuste	R^2	Total proyectado $\mu0$
A. Lineal sobre defectos por sesión	$d = 3,93 - 0,21 \cdot t$	0,05	39,4
B. Log-lineal	pendiente positiva (+0,051)	0,03	descartado: no converge
C. Acumulado saturante	$\mu0 = 57,4$ y $k = 0,071$	0,87	57,4

Se toma el modelo C como estimación central ($\mu0 \approx 57$) y el A como cota baja ($\mu0 \approx 39$).

- **Defectos liberados:** $\mu r = \mu0 - \Sigma\mu i = 57,4 - 30 = 27,4$, es decir unos 27 defectos aún sin encontrar (con el modelo A serían 9,4).
- **Tamaño final:** `cat collatz.py | wc -l` da 45 Locs (la versión v6, con docstrings, validación y `main`; el original tenía 18 según la consigna).
- **Densidad proyectada:** 57,4 / 45 = 1,28 defectos por Loc en total y 27,4 / 45 = 0,61 por Loc de residual (0,21 con el modelo A).

Estas cifras son una estimación estadística, no defectos conocidos: la serie mezcla inspecciones con ejecuciones y no decrece de forma monótona (sube en S6 y S7 porque recién ahí se probó el programa completo), y la versión final pasa los 16 casos.

9. PCE y garantía: comportamiento cualitativo

Subir el PCE reduce los defectos que se entregan, pero cada punto adicional exige más tiempo de test que el anterior, así que el tiempo total de test óptimo crece cada vez más rápido y nunca alcanza el 100 %. Lo razono con el modelo exponencial de detección,

porque no tengo el artículo "Marco para evaluar garantía en desarrollo de software" a mano.

Con μ0 defectos iniciales y tasa de detección k, los defectos remanentes son μr(t) = μ0·e^(−kt) y el PCE es 1 − e^(−kt). Despejando el tiempo:

```
t = \frac{1}{k}\,\ln\frac{1}{1-\mathrm{PCE}}
```

- **Rendimientos decrecientes:** pasar de PCE 90 % a 99 % duplica el tiempo (2,30/k a 4,61/k), y llegar a 100 % requiere tiempo infinito.
- **Efecto sobre la garantía:** los defectos entregados valen μ0·(1 − PCE), por lo que cada mejora del PCE reduce proporcionalmente el riesgo y el costo de garantía.
- **Consecuencia:** para poder otorgar garantía hay que planificar un tiempo de test mayor, y el óptimo está donde el costo marginal de seguir probando iguala el ahorro marginal de garantía.

10. Organización con PCE = 89 %

Con PCE = 89 % y 0,12 defectos por FP en la liberación, un proyecto de 100 FP debería tener unos 109 defectos al terminar la construcción, de los cuales V&V tendría que detectar unos 97. Datos: PCE = 0,89, δr = 0,12 def/FP, S = 100 FP.

Ítem	Cálculo	Resultado
Defectos liberados μr	δr · S = 0,12 · 100	12 defectos
a. Defectos totales esperados μ0	μr / (1 − PCE) = 12 / 0,11	109,1 ≈ 109 defectos
b. Densidad al finalizar la construcción δ0	μ0 / S = 109,1 / 100	1,09 def/FP
c. Defectos a detectar en V&V	μ0 − μr = 0,89 · 109,1	97,1 ≈ 97 defectos

d. Opinión sobre 20 defectos por día en los primeros 3 días

Es probable que el total termine por encima de lo esperable (más de 109) y que se libere con más de 0,12 def/FP si no se amplía el V&V. Los 60 defectos en 3 días ya son el 62 % de los 97 planificados, y a 20 por día el plan se agotaría hacia el día 5.

Si la organización respondiera a su línea de base con un decaimiento exponencial, 60 defectos en 3 días implicarían una tasa k ≈ 0,27 por día, y la detección diaria bajaría de unos 25 a 20 y a 15. Lo observado es plano (20, 20, 20): no hay señal de agotamiento, lo que apunta a un stock de defectos mayor al histórico.

Hay dos explicaciones que los datos no permiten separar: una construcción con más defectos inyectados que la línea de base, o un V&V más efectivo que el habitual. Mientras el ritmo no baje, lo prudente es asumir lo primero, extender el período de V&V y no liberar según el calendario original. Si desde el día 4 la detección cae rápido, la opinión se revisa.

11. Herramienta json2csv y CI/CD

La herramienta quedó terminada y todos los controles de calidad pasan en un ambiente virtual `venv`, pero el workflow de GitHub Actions todavía no se corrió porque falta subir el repositorio. Se ejecuta como `python json2csv.py {-i input.txt} {-o output.txt}`

`{-d delimitador}`: sin `-i` lee stdin, sin `-o` escribe en stdout y el delimitador por defecto es la coma.

Control pedido	Resultado local
venv (a) y esqueleto CookieCutter (b)	`cookiecutter-pypackage` adaptado: paquete `jsonconv`, script raíz `json2csv.py`
ruff (i), black (j), PEP 8 (k)	sin errores (ruff, black y pycodestyle)
PEP 257 (l)	sin errores (pydocstyle y las reglas D de ruff)
MyPy (m) y PyRight (n)	0 errores en modo estricto
Pytest con 85 % o más (o)	37 pruebas exitosas, cobertura 98,6 %
bandit (p) y sin Trufflehog (q)	sin observaciones; Trufflehog no se usa
Documentación con pdoc (r, u)	generada en `docs/api` más `docs/usage.md`
README, CHANGELOG, CONTEXT, requirements, MIT, VERSION y BUILD (c a h, s)	incluidos
Workflow de CI/CD (t)	`.github/workflows/ci.yml` (calidad en Python 3.11 a 3.13) y `docs.yml` (regenera la documentación con cada merge a main)

Manejo de excepciones: JSON inválido (informa línea y columna), estructura no soportada, delimitador inválido, archivo inexistente o directorio, entrada que no es UTF-8, error de escritura y salida previa intacta si la conversión falla. Códigos de salida: 0 correcto, 1 datos, 2 uso, 3 lectura o escritura; los errores van a stderr sin traceback.

Para activarlo en GitHub: crear el repositorio, subir la carpeta (ya tiene un commit local) y abrir un PR para que corra el workflow.

12. Deuda técnica y gestión de calidad

Los factores del proceso de calidad que más ayudan a gestionar la deuda técnica son los que verifican y mantienen la estructura y la claridad del código, porque evitan que la deuda se acumule sin que nadie la vea. No pude acceder a la presentación "Agile and software engineering, an invisible bond", así que me apoyo en un relevamiento de profesionales que estudia justamente esa relación: según Holvitie y otros, las prácticas ágiles que verifican y mantienen la estructura y claridad de los artefactos son las que más reducen la deuda técnica.

Factor gestionado desde calidad	Por qué mejora la gestión de la deuda
Revisiones, inspecciones y estándares de codificación	Detectan temprano los defectos y los problemas de estructura, cuando corregirlos es más barato, y mantienen el código legible
Pruebas automatizadas y de regresión	Permiten refactorizar sin miedo a romper, que es la forma de pagar la deuda

Factor gestionado desde calidad	Por qué mejora la gestión de la deuda
Integración continua con análisis estático y cobertura mínima	Controlan cada cambio, de modo que la calidad no se degrada en silencio entre entregas
Refactorización planificada y definición de terminado con criterios de calidad	Se paga el interés de forma continua en vez de postergarlo
Diseño y arquitectura revisados	Las carencias de arquitectura son, según otro estudio, la causa más frecuente de deuda técnica, y son las más caras de arreglar tarde
Métricas (PCE, densidad de defectos, deuda estimada)	Hacen visible la deuda, y lo que no se ve no se prioriza

Conviene contrastar esta lista con los factores que nombra el artículo de la cátedra.

13. Multa y punto de equilibrio

Aplicar una multa por defecto entregado, o aumentarla, mueve el punto de equilibrio hacia más tiempo de test y un PCE mayor, y los defectos residuales óptimos bajan en proporción inversa a la multa. Uso el modelo simple de la sección 9, porque no tengo el de las clases teóricas: costo total = costo de testear + multa por defectos que quedan.

```
C(t) = c\,t + M\,\mu_0\,e^{-kt}
```

Con c el costo por unidad de tiempo de test, M la multa por defecto entregado, μ0 los defectos iniciales y k la tasa de detección. Derivando e igualando a cero, el óptimo es:

```
t^{*} = \frac{1}{k}\ln\frac{M\,\mu_0\,k}{c}, \qquad \mu_r^{*} = \frac{c}
{M\,k}, \qquad \mathrm{PCE}^{*} = 1-\frac{c}{M\,\mu_0\,k}
```

- **Efecto de la multa:** dt*/dM = 1/(k·M) es positivo, así que más multa significa más test; duplicarla suma ln 2 / k de tiempo y reduce a la mitad los defectos residuales óptimos.
- **Ejemplo** (c = 1, μ0 = 100, k = 0,1): con M = 1 el óptimo es t* = 23,0 y PCE 90 %; con M = 2, 30,0 y 95 %; con M = 5, 39,1 y 98 %; con M = 10, 46,1 y 99 %.
- **Caso límite:** si M·μ0·k ⩽ c, la multa no justifica ni una unidad de test y conviene liberar de inmediato; la multa debe ser lo bastante grande para que garantizar valga la pena.
- **Fundamento:** el punto de equilibrio está donde el costo marginal de seguir probando (c) iguala el ahorro marginal de multa (M·μ0·k·e^(−kt)); la multa multiplica el segundo término y obliga a probar más para igualarlos.

14. Supuestos y limitaciones

- No tuve acceso al notebook `taller_defectos.ipynb`, al tutorial de Kanban, al artículo "Marco para evaluar garantía", a la presentación "Agile and software engineering" ni al modelo de garantía de las clases. Esas respuestas usan métodos estándar (regresión lineal, log-lineal y curva saturante, ley de Little, modelo exponencial) y conviene contrastarlas con el material de la cátedra.

- Las inspecciones no repararon el código, así que cada fase cuenta sus defectos por separado (30 en total). Con causas raíz distintas, el conteo sin duplicados sería de 7 en el código y 5 en los tests.
- PCE de fase = defectos encontrados en la fase dividido por los presentes al entrar; el 100 % es el total observado, como indica la consigna.
- El umbral de "tasa de comentarios mayor a 12" se interpretó como 12 % de líneas de comentario sobre las Locs.
- La interpretación de los defectos de las sesiones de ejecución (un defecto por caso fallido o intento de ejecución fallido) sigue la consigna; los números salen de ejecuciones reales de `run_sesion.py`.
- El workflow de GitHub no se pudo correr desde aquí; los mismos controles se ejecutaron localmente.